Day (69)

* Object oriented principles or Features or concepts (11 days)

Index).

=> What are the Advantage of oops.

=> List of object oriented principles
1) classes
2) objects
3) Data encapsulation
4) Data Abstraction
5) inheritance
6) polymorphism
7) message passing.

=

1) classes
=> importance and purpose of classes concept
=> syntax for Defining class
=> Types of Data members
a) instance Data members.
b) class level Data members.
=> Types of methods.
a) instance methods.
b) class level methods.
c) static methods.
=> what is "self" & "cls"
=> programming examples.

2) object
=> importance and purpose of object concept
=> syntax for creating object
=> Examples.

- programming examples related to pickling & Database communication with classes and approach.

=> constructors in oops
=> importance and purpose
=> Types of constructor
a) Default
b) parameterized
=> Rules for using constructors
=> programming example.

=> Destructors in oops with Garbage collector
=> importance & purpose of Destructor
=> syntax.
=> internal flow
=> relationship B/w Destructor & Garbage Collector
=> gc module
=> examples

3) Data Encapsulation & 4) Data Abstraction
- => importance & purpose
- => implementation
- => programming examples.

5) inheritance
- => importance and purpose of inheritance
- => Types of inheritances.
  - a) single
  - b) multilevel
  - c) hierarchical
  - d) multiple
  - e) Hybrid
- => syntax
- => examples.

=> Method overriding in oops
- => importance & purpose
- => memory management
- => examples.

=> Constructor overriding in oops.
- => importance and purpose of constructor overriding
- => memory management in
- => examples

6) polymorphism
- => importance & purpose
- => Diff. b/w polymorphism & inheritance
- => method overriding.
- => constructor overriding
- => super() and class name approaches in polymorphism
- => examples

* Introduction to object oriented principles or features or Concepts

→ In Real time, to develop any projects, we need to choose a Language.

→ The Language, which we select for developing the project can satisfy two types of principles.

They are:

1) procedural or Functional oriented principles
2) object oriented principles.

→ In otherwords, in industry, we have two types of programming Languages. They are:

1) procedural OR Functional oriented programming Languages.

→ if programming Language satisfies procedure OR functional oriented principles then that Langvages are called procedure OR functional oriented programming Languages.

→ examples) C, pascal, COBOL, Python, upto oracle 7.3 .. etc.

2) object oriented programming Languages:-

→ if programming Language satisfies object oriented principles then that Langvages are called object oriented programming Language.

→ examples) Lisp, small talk, Ruby, C++, Java, C#.net, Python oracle 8 onwards etc.

⇒ Even though Python Belongs to Both functional and object oriented programming Languages, internally Every thing treated as object.

Every thing in python is an object --- Advantage (OR)

Advantages of oops

1) it stores unlimited amount of Data
2) The large of volume of Data Transfered between multiple remote machines all at once gets effective Communication.

3) The confidential Data transfered between multiple remote machines in the form of cipher text. So that security enhanced (improved).

4) The Data always available around the objects and gives less memory

5) We can build high and Re-usable Applications by using inheritance.

* object oriented principles or features or concepts in Python.

→ To say a programming language is object-oriented then it has to satisfy the following principles.

1) Classes
2) objects
3) Data Encapsulation
4) Data Abstraction
5) inheritance.
6) polymorphism.
7) Message passing (already discussed)

Day 70

1) classes :-

→ The purpose of classes concept is that "To develop programmer-Defined data type + To develop any real time Application.

→ The purpose of Developing programmer-Defined Data Type is that "to store customized Data and to perform customized operations".

→ To Develop programmer-Defined Data Type with classes concept, we use 'class' keyword.

→ programatically, All programmers-Defined class Names are treated as programmer-Defined Data Type.

→ In oops, Every Python program must starts with classes concept (without classes concept there is no program).

*Definition of class :-

→ A class is collection of Data members (properties) and methods (Behaviours -- functions)

→ When we define a class, there is no memory space created for Data members and methods of a class But whose memory space created when we create an object w.r.t class.

→ Whatever Data members and methods we specify inside the class definition, they are reflected or appeared to the object with memory space. So that object contains Data members for storing the Data and object contains methods for performing operations on the Data members of corresponding object

Syntax for Defining the class in python →

```
Class <clsname>:
        class level Data Members
        def instance method name (self, list of formal params if any):
                Block of statements perform specific operations
                on objects specify instance Data members.

        @classmethod
        def classlevelmethod ( cls, list of formal params if any):
                Block of stmnts perform common operations
                of same class specify class level
                Data members.

        def staticmethodname (list of formal params if any):
                Block of stmnt performs universal/
                utility operations.

        def __init__ (self, list of formal params if any):
                Block of stmnt which will initialize the
                object of same class.
```

def. __del__(self):
Block of stmnt which will be deallocate the memory space related with garbage Collector.

* Types of Data members:-

→ In a class of python, we can Define two types of Data members. They are:

1) instance Data members.
2) class level Data members.

1) instance Data members:-

→ instance Data members are those which are used for storing specific values.

→ instance Data members memory space to be created Each and every time when an object is created.

→ programatically, we can Define or specify the instance Data members in 3 ways.

They are: 1) Through an object
2) inside of instance method
3) inside of constructor.

→ instance Data members must be Accessed either w.r.t object name or self

syntax 1) objectname.Instance Data Member Name

syntax 2) self.Instance Data member Name (use this syntax in inside of instance method)

2) class level Data members:-

→ class level Data members are those which are used for storing common values.

→ class level Data members memory space created only once irrespective of number of object created of same class.

→ Programatically, we can define or specify the class level data members in 2 ways.

They are:

1) inside the class definition
2) inside the class level method.

→ class level data members can be Accessed either w.r.t class name or object Name or cls or self

syntax 1) ClassName.class level Data member name

syntax 2) objectName.Class level Data member name

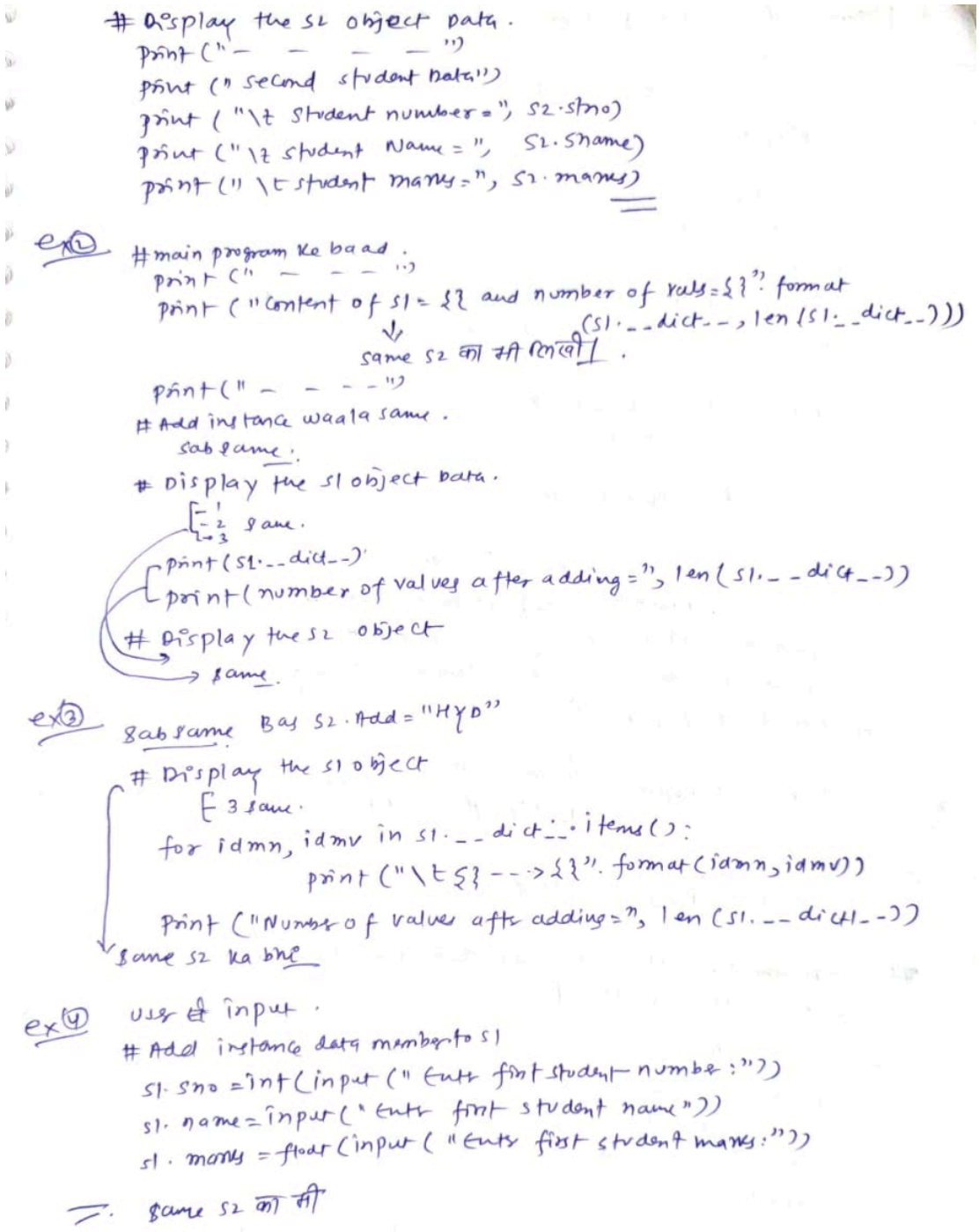
syntax 3) cls.class level Data member name (use this syntax in inside of Class level method)

syntax 4) self.class level Data member name (use this syntax in inside of instance method)

program for demonstrating instance Data members.

ex①

```
Class student: pass
#main program
s1 = student()   # object creation
s2 = student()   # object creation
# Add instance Data members to s1 object by using OBJECT itself
s1.sno = 100
s1.sname = "Rg"
s1.marks = 44.56.
# Add instance Data members to s2 object by using Object itself
s2.sno = 200
s2.sname = "Hf"
s2.marks = 50.57
# Display the s1 object data
print(_____)
print("first student Data")
print("-------")
print("\t student Number =", s1.sno)
print("\t student name =", s1.name)
print("\t student marks =", s1.marks).
```



In [1]:
```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks
#InstanceDataMembersEx1
class Student:pass

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=100
s1.name="RS"
s1.marks=44.56
#Add Instance Data Members to s2 Object by using OBJECT Itself
```

```
s2.stno=200
s2.sname="TR"
s2.marks=56.78
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-------------------------------------")
print("First Student Data")
print("-------------------------------------")
print("\tStudent Number=",s1.sno)
print("\tStudent Name=",s1.name)
print("\tStudent Marks=",s1.marks)
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-------------------------------------")
print("Second Student Data")
print("\tStudent Number=",s2.stno)
print("\tStudent Name=",s2.sname)
print("\tStudent Marks=",s2.marks)
print("-------------------------------------")
```

```
-------------------------------------
First Student Data
-------------------------------------
        Student Number= 100
        Student Name= RS
        Student Marks= 44.56
-------------------------------------
Second Student Data
        Student Number= 200
        Student Name= TR
        Student Marks= 56.78
-------------------------------------
```

In [2]:

```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks
#InstanceDataMembersEx2
class Student:pass

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-------------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-------------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=100
s1.name="RS"
s1.marks=44.56
#Add Instance Data Members to s2 Object by using OBJECT Itself
s2.stno=200
s2.sname="TR"
s2.marks=56.78
s2.addr="HYD"
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-------------------------------------")
print("First Student Data")
print("-------------------------------------")
print(s1.__dict__)
print("Number of Values after adding=",len(s1.__dict__))
print("-------------------------------------")
print("Second Student Data")
```

```
print("-----------------------------------")
print(s2.__dict__)
print("Number of Values after adding=",len(s2.__dict__))
print("-----------------------------------")
```

```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
-----------------------------------
First Student Data
-----------------------------------
{'sno': 100, 'name': 'RS', 'marks': 44.56}
Number of Values after adding= 3
-----------------------------------
Second Student Data
-----------------------------------
{'stno': 200, 'sname': 'TR', 'marks': 56.78, 'addr': 'HYD'}
Number of Values after adding= 4
-----------------------------------
```

In [3]:
```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks
#InstanceDataMembersEx3
class Student:pass

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-----------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-----------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=100
s1.name="RS"
s1.marks=44.56
#Add Instance Data Members to s2 Object by using OBJECT Itself
s2.stno=200
s2.sname="TR"
s2.marks=56.78
s2.addr="HYD"
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-----------------------------------")
print("First Student Data")
print("-----------------------------------")
for idmn,idmv in s1.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Number of Values after adding=",len(s1.__dict__))
print("-----------------------------------")
print("Second Student Data")
print("-----------------------------------")
for idmn,idmv in s2.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Number of Values after adding=",len(s2.__dict__))
print("-----------------------------------")
```

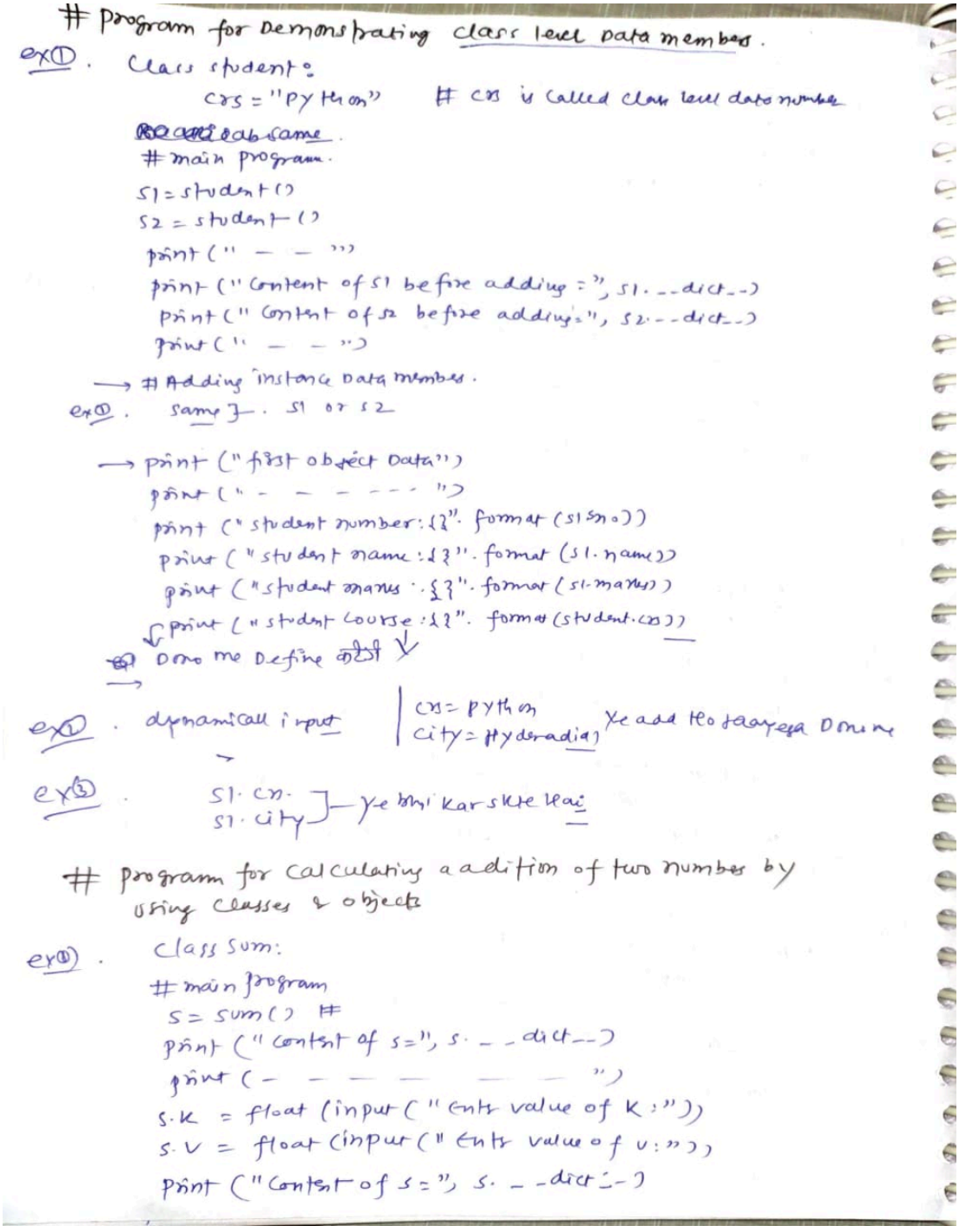
```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
-----------------------------------
First Student Data
-----------------------------------
        sno--->100
        name--->RS
        marks--->44.56
Number of Values after adding= 3
-----------------------------------
Second Student Data
-----------------------------------
        stno--->200
        sname--->TR
        marks--->56.78
        addr--->HYD
Number of Values after adding= 4
-----------------------------------
```

In [4]:
```python
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks
#InstanceDataMembersEx4
class Student:pass

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-----------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-----------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=int(input("Enter First Student Number:"))
s1.name=input("Enter First Student Name:")
s1.marks=float(input("Enter First Student Marks:"))
#Add Instance Data Members to s2 Object by using OBJECT Itself
s2.sno=int(input("Enter second Student Number:"))
s2.name=input("Enter Second Student Name:")
s2.marks=float(input("Enter Second Student Marks:"))
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-----------------------------------")
print("First Student Data")
print("-----------------------------------")
for idmn,idmv in s1.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Number of Values after adding=",len(s1.__dict__))
print("-----------------------------------")
print("Second Student Data")
print("-----------------------------------")
for idmn,idmv in s2.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Number of Values after adding=",len(s2.__dict__))
print("-----------------------------------")
```

```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
```

```
-----------------------------------
First Student Data
-----------------------------------
        sno--->50
        name--->arif
        marks--->500.0
Number of Values after adding= 3
-----------------------------------
Second Student Data
-----------------------------------
        sno--->60
        name--->raza
        marks--->450.0
Number of Values after adding= 3
-----------------------------------
```



In [5]:
```
#program for Demonstrating Class Level Data Members
#ClassLevelDataMemberEx1
class Student:
    crs="PYTHON" # here crs is called Class Level Data Member
class Emp:
    compname="IBM" # here compname is called Class Level Data Member
#Main Program
print("Course for all Students=",Student.crs)
print("Comp Name of all Employees=",Emp.compname)
print("------------------OR------------------")
s=Student()
e=Emp()
print("Course for all Students=",s.crs)
print("Comp Name of all Employees=",e.compname)
```

```
Course for all Students= PYTHON
Comp Name of all Employees= IBM
------------------OR------------------
Course for all Students= PYTHON
Comp Name of all Employees= IBM
```

In [6]:
```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks and course
#ClassLevelDataMemberEx2
class Student:
    crs="PYTHON PROGRAMMING" # here cls is called Class Level Data Member

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-------------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-------------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=int(input("Enter First Student Number:"))
s1.name=input("Enter First Student Name:")
s1.marks=float(input("Enter First Student Marks:"))
#Add Instance Data Members to s2 Object by using OBJECT Itself
s2.sno=int(input("Enter second Student Number:"))
s2.name=input("Enter Second Student Name:")
s2.marks=float(input("Enter Second Student Marks:"))
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-------------------------------------")
print("First Student Data")
print("-------------------------------------")
for idmn,idmv in s1.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Course Name for all Student=",Student.crs)
print("Number of Values after adding=",len(s1.__dict__))
print("-------------------------------------")
print("Second Student Data")
print("-------------------------------------")
for idmn,idmv in s2.__dict__.items():
    print("\t{}--->{}".format(idmn,idmv))
print("Course Name for all Student=",Student.crs)
print("Number of Values after adding=",len(s2.__dict__))
print("-------------------------------------")
```

```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
-----------------------------------
First Student Data
-----------------------------------
        sno--->50
        name--->arif
        marks--->500.0
Course Name for all Student= PYTHON PROGRAMMING
Number of Values after adding= 3
-----------------------------------
Second Student Data
-----------------------------------
        sno--->60
        name--->raza
        marks--->450.0
Course Name for all Student= PYTHON PROGRAMMING
Number of Values after adding= 3
-----------------------------------
```

In [7]:

```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks and course
#ClassLevelDataMemberEx3
class Student:
    crs="PYTHON PROGRAMMING" # here cls is called Class Level Data Member

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-----------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-----------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
s1.sno=int(input("Enter First Student Number:"))
s1.name=input("Enter First Student Name:")
s1.marks=float(input("Enter First Student Marks:"))
#Add Instance Data Members to s2 Object by using OBJECT Itself
s2.sno=int(input("Enter second Student Number:"))
s2.name=input("Enter Second Student Name:")
s2.marks=float(input("Enter Second Student Marks:"))
#Display the S1 Object Data--Instance Data Members of s1 Object
print("-----------------------------------")
print("First Student Data")
print("-----------------------------------")
print("\tStudent Number=",s1.sno)
print("\tStudent Name=",s1.name)
print("\tStudent Marks=",s1.marks)
print("\tSTUDENT COURSE=",Student.crs)
print("-----------------------------------")
print("Second Student Data")
print("-----------------------------------")
print("\tStudent Number=",s2.sno)
print("\tStudent Name=",s2.name)
print("\tStudent Marks=",s2.marks)
print("\tSTUDENT COURSE=",Student.crs)
print("-----------------------------------")
```

```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
-----------------------------------
First Student Data
-----------------------------------
        Student Number= 10
        Student Name= arif
        Student Marks= 450.0
        STUDENT COURSE= PYTHON PROGRAMMING
-----------------------------------
Second Student Data
-----------------------------------
        Student Number= 20
        Student Name= raza
        Student Marks= 456.0
        STUDENT COURSE= PYTHON PROGRAMMING
-----------------------------------
```

In [8]:

```
#Program for Demonstrating Instance Data Members
#I want to Store sno,sname and Marks and course
#ClassLevelDataMemberEx4
class Student:
    crs="PYTHON PROGRAMMING" # here cls is called Class Level Data Member

#Main Program
s1=Student() # Object Creation
s2=Student() # Object Creation
print("-----------------------------------")
print("Content of s1={} and Number of Vals={}".format(s1.__dict__,len(s1.__dict_
print("Content of s2={} and Number of Vals={}".format(s2.__dict__,len(s2.__dict_
print("-----------------------------------")
#Add Instance Data Members to s1 Object by using OBJECT Itself
while(True):
    try:
        s1.sno=int(input("Enter First Student Number:"))
        s1.name=input("Enter First Student Name:")
        s1.marks=float(input("Enter First Student Marks:"))
    except ValueError:
        print("Don't Enter alnums,strs and synbols for sno and marks")
    else:
        # Display the S1 Object Data--Instance Data Members of s1 Object
        print("-----------------------------------")
        print("First Student Data")
        print("-----------------------------------")
        print("\tStudent Number=", s1.sno)
        print("\tStudent Name=", s1.name)
        print("\tStudent Marks=", s1.marks)
        print("\tSTUDENT COURSE=", s1.crs)
        print("-----------------------------------")
        break
#Add Instance Data Members to s2 Object by using OBJECT Itself
while(True):
    try:
        s2.sno=int(input("Enter second Student Number:"))
        s2.name=input("Enter Second Student Name:")
        s2.marks=float(input("Enter Second Student Marks:"))
    except ValueError:
        print("Don't Enter alnums,strs and synbols for sno and marks")
```

```
    else:
        print("Second Student Data")
        print("-----------------------------------")
        print("\tStudent Number=",s2.sno)
        print("\tStudent Name=",s2.name)
        print("\tStudent Marks=",s2.marks)
        print("\tSTUDENT COURSE=",s2.crs)
        print("-----------------------------------")
        break
```

```
-----------------------------------
Content of s1={} and Number of Vals=0
Content of s2={} and Number of Vals=0
-----------------------------------
-----------------------------------
First Student Data
-----------------------------------
        Student Number= 20
        Student Name= arif
        Student Marks= 450.0
        STUDENT COURSE= PYTHON PROGRAMMING
-----------------------------------
Second Student Data
-----------------------------------
        Student Number= 21
        Student Name= raza
        Student Marks= 451.0
        STUDENT COURSE= PYTHON PROGRAMMING
-----------------------------------
```

```
print ("- - - - - - - - ")
# Add the values.
s.r = s.k + s.v
print ("- - - - - ")
print ("first value: {} ": format (s.k))
      second value          (s.v)
      sum                   (s.r)
```

* Day (71)

* Types of methods in class of python:-

→ In a class of python programming, we can define 3 types of methods. They are:-

1) instance methods.
2) class level methods.
3) static methods.

1) instance methods:-

→ instance methods are those which are used for performing specific operations on objects and Hence instance methods are also called object level methods.

→ syntax for defining instance method is

```
def instancemethodname (self, list of formal params if any):
        specify instance data members
        perform specific operations.
```

→ instance methods must accessed w.r.t object name or self.

- objectname.InstanceMethodName()
  (OR)
- self.InstanceMethodName() (To be used inside of instance method only)

=> What is 'self'?-

→ "self" is one of the implicit object and it contains Address of current object.

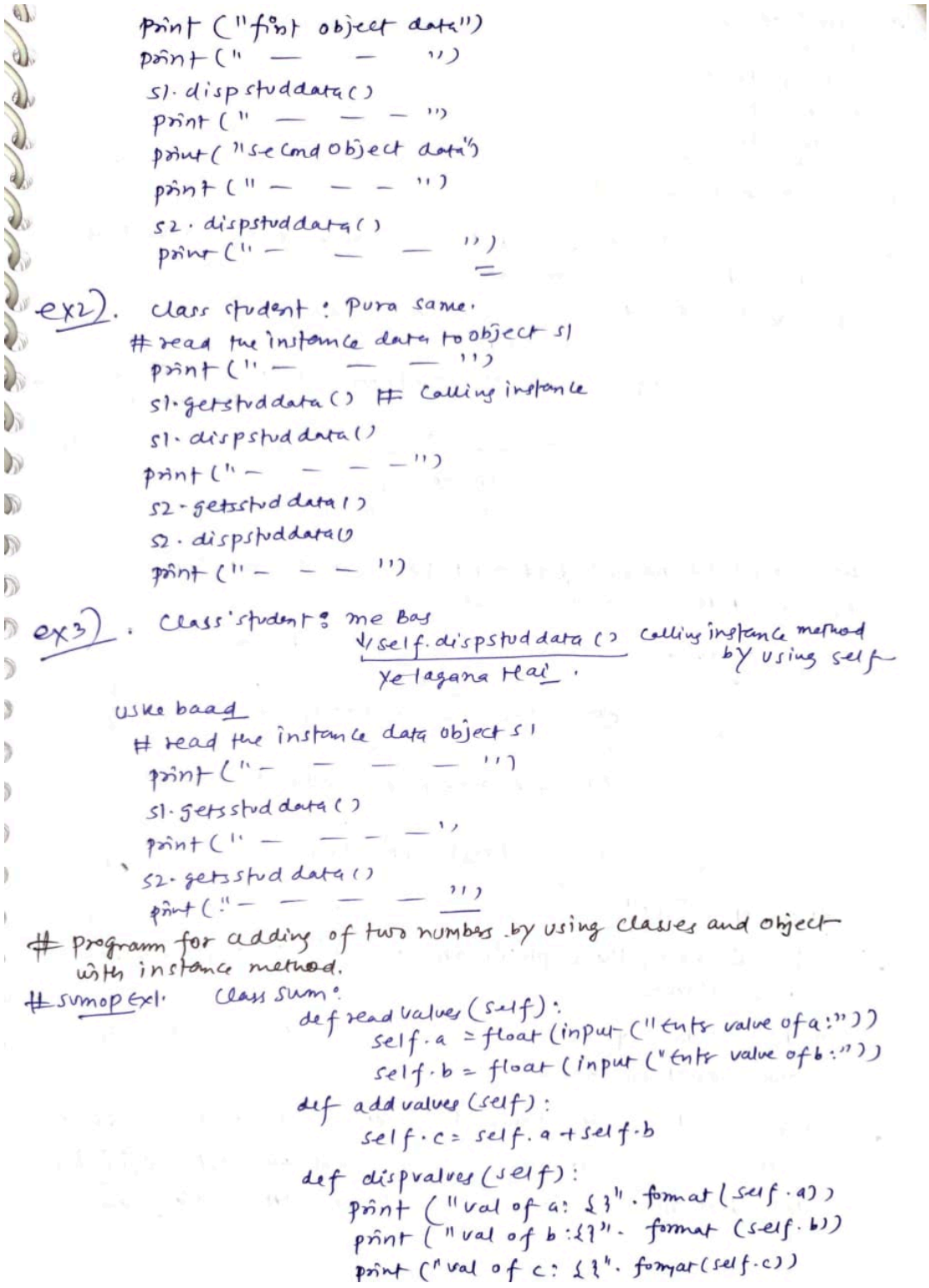
→ "self" always to be used as first formal parameter in instance method.

→ since "self" is a formal parameter, so that it can access inside of corresponding instance method Definition only but not possible to access other part of program.

Program for demonstrating instance method:-

ex1) class student:
    def getstuddata(self): #instance method
        print("-" * 50)
        self.sno = int(input("Ents student number:", id(self))

```
class student:
    def getstuddata(self): # instance method.
        print("ID of current in getstuddata()", id(self))
        self.sno = int(input("Ents student number"))
        self.sname = input("Ents student name:")
        self.marks = float(input("Ents student marks:"))
    def getsstuddata(self):
        print("student number:{}".format(self.sno))
        print("student Name:{}".format(self.sname))
        print("student marks:{}".format(self.marks))
#main programs
s1 = student()
s2 = student()
#read the instance data to object s1
print("---------")
print("ID of s1 in main program =", id(s1))
s1.getstuddata()  # Calling instance method w.r.t object name
print("---------")
print("ID of s2 in main programs =", id(s2))
s2.getstuddata()  #calling instance method.
print("---------").
```



In [9]:
```
#Program for Demonstrating Instance Method
#InstanceMethodEx1
class Student:
    def getstuddata(self,objinfo):
        print("-"*50)
        self.sno=int(input("Enter {} Student Number:".format(objinfo)))
        self.name=input("Enter {} Student Name:".format(objinfo))
        self.marks=float(input("Enter {} Student Marks:".format(objinfo)))
        print("-" * 50)
    def dispstuddata(self,objinfo):
        print("-"*50)
        print("{} Student Details".format(objinfo))
        print("-" * 50)
```

```
        print("\tStudent Number:{}".format(self.sno))
        print("\tStudent Name:{}".format(self.name))
        print("\tStudent Marks:{}".format(self.marks))
        print("-" * 50)
#Main Program
s1=Student()
s2=Student()
print("Id of s1 in main program=",id(s1))
print("Id of s2 in main program=",id(s2))
print("------------------------")
s1.getstuddata("First")
s2.getstuddata("Second")
s1.dispstuddata("First")
s2.dispstuddata("Second")
```

```
Id of s1 in main program= 2345606068752
Id of s2 in main program= 2345630290704
------------------------
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
First Student Details
--------------------------------------------------
        Student Number:40
        Student Name:arif
        Student Marks:480.0
--------------------------------------------------
--------------------------------------------------
Second Student Details
--------------------------------------------------
        Student Number:50
        Student Name:raza
        Student Marks:500.0
--------------------------------------------------
```

In [10]:
```
#Program for Demonstrating Instance Method
#InstanceMethodEx2
class Student:
    def getstuddata(self,objinfo):
        print("-"*50)
        self.sno=int(input("Enter {} Student Number:".format(objinfo)))
        self.name=input("Enter {} Student Name:".format(objinfo))
        self.marks=float(input("Enter {} Student Marks:".format(objinfo)))
        print("-" * 50)
        self.dispstuddata(objinfo)#Calling instance method w.r.t self
    def dispstuddata(self,objinfo):
        print("-"*50)
        print("{} Student Details".format(objinfo))
        print("-" * 50)
        print("\tStudent Number:{}".format(self.sno))
        print("\tStudent Name:{}".format(self.name))
        print("\tStudent Marks:{}".format(self.marks))
        print("-" * 50)
#Main Program
s1=Student()
s2=Student()
#read and display First Student Data
```

```
s1.getstuddata("First")
s2.getstuddata("Second")
```

```
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
First Student Details
--------------------------------------------------
        Student Number:40
        Student Name:arif
        Student Marks:450.0
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
Second Student Details
--------------------------------------------------
        Student Number:41
        Student Name:raza
        Student Marks:452.0
--------------------------------------------------
```

In [11]:
```
#Program for Demonstrating Instance Method
#InstanceMethodEx3:-
class Student:
    crs="PYTHON" # Here crs is called Class Level Data Member
    def getstuddata(self,objinfo):
        print("-"*50)
        self.sno=int(input("Enter {} Student Number:".format(objinfo)))
        self.name=input("Enter {} Student Name:".format(objinfo))
        self.marks=float(input("Enter {} Student Marks:".format(objinfo)))
        print("-" * 50)
        self.dispstuddata(objinfo)#Calling instance method w.r.t self
    def dispstuddata(self,objinfo):
        print("-"*50)
        print("{} Student Details".format(objinfo))
        print("-" * 50)
        print("\tStudent Number:{}".format(self.sno))
        print("\tStudent Name:{}".format(self.name))
        print("\tStudent Marks:{}".format(self.marks))
        print("\tSTUDENT COURSE:{}".format(self.crs))
        #OR print("\tSTUDENT COURSE:{}".format(Student.crs))
        print("-" * 50)
#Main Program
s1=Student()
s2=Student()
#read and display First Student Data
s1.getstuddata("First")
s2.getstuddata("Second")
```

```
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
First Student Details
--------------------------------------------------
        Student Number:40
        Student Name:arif
        Student Marks:450.0
        STUDENT COURSE:PYTHON
--------------------------------------------------
--------------------------------------------------
```

```
--------------------------------------------------
--------------------------------------------------
Second Student Details
--------------------------------------------------
        Student Number:40
        Student Name:raza
        Student Marks:456.0
        STUDENT COURSE:PYTHON
--------------------------------------------------
```

In [12]:
```python
#Program for adding of two Numbrs by using Classes and Objects
#instancemethod EX4:-
class SumOp:
    def getvals(self):
        self.a=float(input("Enter First value:"))
        self.b=float(input("Enter Second Value:"))
    def calculate(self):
        self.c=self.a+self.b
    def dispvals(self):
        print("sum({},{})={}".format(self.a,self.b,self.c))

#Main Program
so=SumOp()
so.getvals()
so.calculate()
so.dispvals()
```

```
sum(50.0,40.0)=90.0
```

#main program
s1=sum()
s1.readvalues()
s1.addvalues()
s1.dispvalues()

2.) Class level methods :-

→ Class level methods are used for performing common operation on all objects of same class

→ The syntax for defining class level method is

@classmethod
def classlevelmethodname(cls, list of formal params if any):
— — — — — — — — —
Performs common operations for the objects of same class,
specify class level Data members.

→ All class level method must be accessed w.r.t Class Name or object Name or cls or self

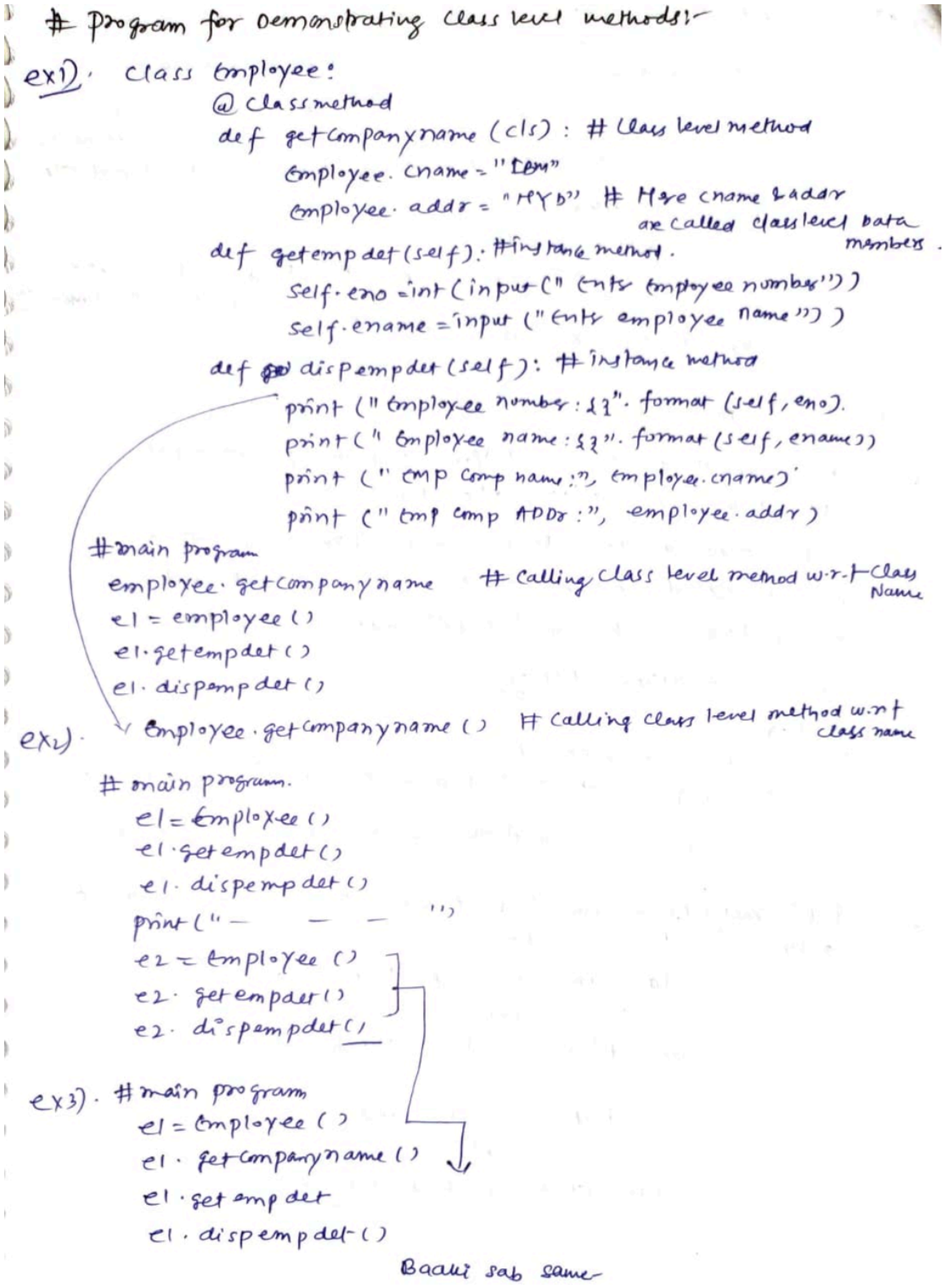
syntax: clsname.classlevelmethodName()
(or)
cls.classlevelMethodName()
(or)
objectname.classlevelmethodName()
(or)
self.classlevelMethodName()

#what is "cls"

→ "cls" is one of the implicit object and it contains current class Name.

→ "cls" always to be used as First Formal parameter in class level method.

→ since "cls" is a formal parameter, so that it can access inside of corresponding class level method definition only but not possible to access other part of program.



In [13]:
```
#Program for Demonstrating Class Level Method
#ClassLevelMethodEx1
class Student:
   @classmethod
   def getcrs(cls):
       Student.crs = "Python"  # Class Level Data Member
       cls.addr="HYDERABAD"  # Class Level Data Member
#Main Program
Student.getcrs() # Calling Instance Method w.r.t Class Name
print("Student Course=",Student.crs)
print("Student Addr=",Student.addr)
```

```
Student Course= Python
Student Addr= HYDERABAD
```

In [14]:
```
#Program for Demonstrating Class Level Method
#ClassLevelMethodEx2
class Student:
   @classmethod
   def getcrs(cls):
       Student.crs = "Python"  # Class Level Data Member
   @classmethod
   def getaddr(cls):
       cls.addr="HYDERABAD"  # Class Level Data Member
#Main Program
Student.getcrs() # Calling Instance Method w.r.t Class Name
Student.getaddr() # Calling Instance Method w.r.t Class Name
print("Student Course=",Student.crs)
print("Student Addr=",Student.addr)
```

```
Student Course= Python
Student Addr= HYDERABAD
```

In [15]:
```
#Program for Demonstrating Class Level Method
#ClassLevelMethodEx3
class Student:
   @classmethod
   def getcrs(cls):
       Student.crs = "Python"  # Class Level Data Member
   @classmethod
   def getaddr(cls):
       Student.getcrs() # Calling Class Level method w.r.t Class Name
       cls.addr="HYDERABAD"  # Class Level Data Member
#Main Program
Student.getaddr() # Calling Instance Method w.r.t Class Name
print("Student Course=",Student.crs)
print("Student Addr=",Student.addr)
```

```
Student Course= Python
Student Addr= HYDERABAD
```

In [16]:
```
#Program for Demonstrating Class Level Method
#ClassLevelMethodEx4
class Student:
   @classmethod
   def getcrs(cls):
       Student.crs = "Python"  # Class Level Data Member
   @classmethod
   def getaddr(cls):
       cls.getcrs() # Calling Class Level method w.r.t cls
       cls.addr="HYDERABAD"  # Class Level Data Member
#Main Program
Student.getaddr() # Calling Instance Method w.r.t Class Name
print("Student Course=",Student.crs)
print("Student Addr=",Student.addr)
```

```
Student Course= Python
Student Addr= HYDERABAD
```

ex 4). class employee:

@classmethod
def getcompanyname (cls) : # class level method
cls.cname = "IBM" # Here cname is called class level data member
cls.getaddr() # Calling class level method w.r.t cls.

@classmethod
def getaddr(cls):
cls.addr = "HYD" # Here Addr is called class level Data member

Baaki sab ex(1) की तरह लिख दीजिए।

3.) static methods :-

→ static methods are used for performing universal operations or utility operations.

→ static methods definition must be preceded with a predefined decorator called @staticmethod and it never takes "cls" or "self" but always takes object of other classes.

→ syntax

@staticmethod
def staticmethodname (list of formal params):
utility operation / universal operation

→ static methods can be accessed w.r.t class name or object name or cls or self:-

ClassName.static method name()
(OR)
objectName.static method name()
(OR)
cls.static method name()
(OR)
self.static method name()

program for demonstrating static method

ex1)

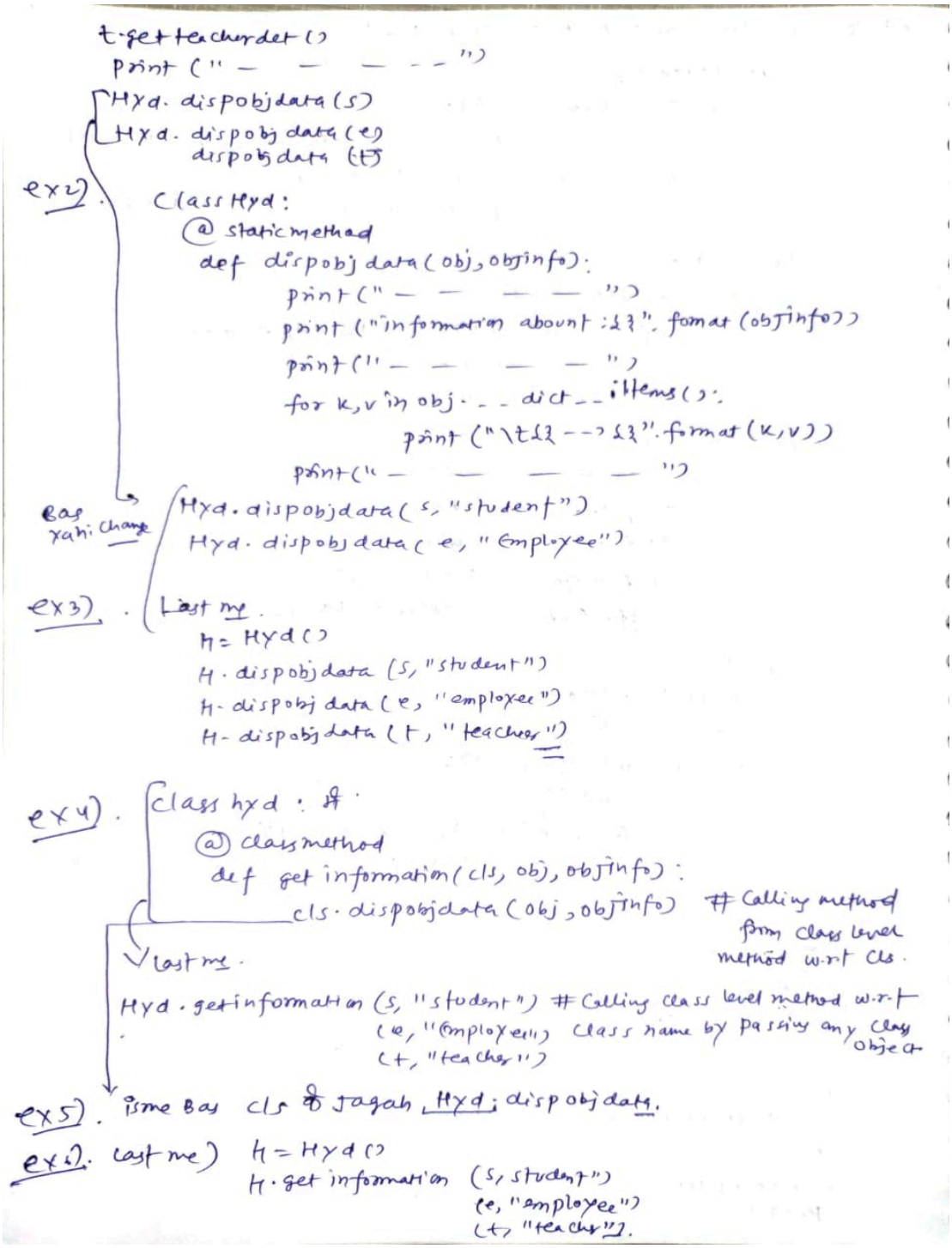
```
class student:
    def getstuddata(self):
        self.sno = int(input("Enter student number:"))
        self.name = input("Enter student Name:")
        self.marks = float(input("Enter student marks:"))

class employee:
    def getstuddata(self):
        self.eno = int(input("Enter employee numbe
        self.ename = name
        self.sal = salary.

class Teacher:
    def getteacherdata(self):
        self.tno = number
        self.name = name
        self.marks = Teacher subject.
        self.exp = Teacher experience

class Hyd:
    @staticmethod
    def disobjectdata(objdata,objinfo)
    def dispobjdata(obj):
        print("- - - -")
        for k,v in obj.__dict__.items():
            print("\t{} ---> {}".format(k,v))
        print("- - - -").

#main program
s=student()
e=employee()
t=Teacher()
print("- - - -")
s.getsstuddata()
print("- - - -")
e.getempdata()
print("- - - -")
```



In [17]:
```
#Program for Demonstrating Static Method
#StaticMethodEx1
class Student:
    def getstuddata(self):
        self.sno=int(input("Enter Student  Number:"))
        self.name =input("Enter Student  Name:")
        self.marks=float(input("Enter Student  Marks:"))
class Employee:
    def getempdata(self):
        self.eno=int(input("Enter Employee  Number:"))
        self.name =input("Enter Employee  Name:")
        self.sal=float(input("Enter Employee Salary:"))
class Teacher:
    def getteacherdata(self):
        self.tno=int(input("Enter Teacher  Number:"))
```

```
        self.name =input("Enter Teacher Name:")
        self.marks=input("Enter Teacher  Subject:")
        self.exp=int(input("Enter Teacher  Exp:"))
class Hyd:
    @staticmethod
    def dispobjectdata(objdata,objinfo):
        print("-"*50)
        print("{} Object Information".format(objinfo))
        print("-" * 50)
        for k,v in objdata.__dict__.items():
            print("\t{}--->{}".format(k,v))
        print("-"*50)

#Main Program
s=Student()
e=Employee()
t=Teacher()
s.getstuddata()
print("---------------------------------")
e.getempdata()
print("---------------------------------")
t.getteacherdata()
#Calling Static Method of Class
Hyd.dispobjectdata(s,"Student") # Calling static Method w.r.t Class Name
Hyd.dispobjectdata(e,"Employee") # Calling static Method w.r.t Class Name
Hyd.dispobjectdata(t,"Teacher") # Calling static Method w.r.t Class Name
```

```
---------------------------------
---------------------------------
--------------------------------------------------
Student Object Information
--------------------------------------------------
        sno--->40
        name--->arif
        marks--->567.0
--------------------------------------------------
--------------------------------------------------
Employee Object Information
--------------------------------------------------
        eno--->45
        name--->raza
        sal--->35000.0
--------------------------------------------------
--------------------------------------------------
Teacher Object Information
--------------------------------------------------
        tno--->23
        name--->md
        marks--->maths
        exp--->12
--------------------------------------------------
```

In [18]:
```
#Program for Demonstrating Static Method
#StaticMethodEx2
class Student:
    def getstuddata(self):
        self.sno=int(input("Enter Student  Number:"))
        self.name =input("Enter Student  Name:")
        self.marks=float(input("Enter Student  Marks:"))
class Employee:
```

```
    def getempdata(self):
        self.eno=int(input("Enter Employee  Number:"))
        self.name =input("Enter Employee  Name:")
        self.sal=float(input("Enter Employee Salary:"))
class Teacher:
    def getteacherdata(self):
        self.tno=int(input("Enter Teacher  Number:"))
        self.name =input("Enter Teacher Name:")
        self.marks=input("Enter Teacher  Subject:")
        self.exp=int(input("Enter Teacher  Exp:"))
class Hyd:
    @staticmethod
    def dispobjectdata(objdata,objinfo):
        print("-"*50)
        print("{} Object Information".format(objinfo))
        print("-" * 50)
        for k,v in objdata.__dict__.items():
            print("\t{}--->{}".format(k,v))
        print("-"*50)

#Main Program
s=Student()
e=Employee()
t=Teacher()
s.getstuddata()
print("---------------------------------")
e.getempdata()
print("---------------------------------")
t.getteacherdata()
#Calling Static Method of Class
h=Hyd()
h.dispobjectdata(s,"Student") # Calling static Method w.r.t Object Name
h.dispobjectdata(e,"Employee") # Calling static Method w.r.t Object Name
h.dispobjectdata(t,"Teacher") # Calling static Method w.r.t Class Name
```

```
---------------------------------
---------------------------------
--------------------------------------------------
Student Object Information
--------------------------------------------------
        sno--->30
        name--->arif
        marks--->430.0
--------------------------------------------------
--------------------------------------------------
Employee Object Information
--------------------------------------------------
        eno--->34
        name--->raza
        sal--->19000.0
--------------------------------------------------
--------------------------------------------------
Teacher Object Information
--------------------------------------------------
        tno--->24
        name--->kvr
        marks--->20
        exp--->20
--------------------------------------------------
```

In [19]:
```
#Program for Demonstrating Static Method
#StaticMethodEx3
class Student:
    def getstuddata(self):
        self.sno=int(input("Enter Student  Number:"))
        self.name =input("Enter Student  Name:")
        self.marks=float(input("Enter Student  Marks:"))
class Employee:
    def getempdata(self):
        self.eno=int(input("Enter Employee  Number:"))
        self.name =input("Enter Employee  Name:")
        self.sal=float(input("Enter Employee Salary:"))
class Teacher:
    def getteacherdata(self):
        self.tno=int(input("Enter Teacher  Number:"))
        self.name =input("Enter Teacher Name:")
        self.marks=input("Enter Teacher  Subject:")
        self.exp=int(input("Enter Teacher  Exp:"))
class Hyd:
    @classmethod
    def getobjectdata(cls,objdata,objinfo):
        cls.dispobjectdata(objdata,objinfo) # Calling static Method w.r.t cls

    @staticmethod
    def dispobjectdata(objdata,objinfo):
        print("-"*50)
        print("{} Object Information".format(objinfo))
        print("-" * 50)
        for k,v in objdata.__dict__.items():
            print("\t{}--->{}".format(k,v))
        print("-"*50)

#Main Program
s=Student()
e=Employee()
t=Teacher()
s.getstuddata()
print("---------------------------------")
e.getempdata()
print("---------------------------------")
t.getteacherdata()
#Calling Class Method of Class
Hyd.getobjectdata(s,"Student") # Calling static Method w.r.t Class Name
Hyd.getobjectdata(e,"Employee") # Calling static Method w.r.t Class Name
Hyd.getobjectdata(t,"Teacher") # Calling static Method w.r.t Class Name
```

```
---------------------------------
---------------------------------
```

```
--------------------------------------------------
Student Object Information
--------------------------------------------------
        sno--->20
        name--->arif
        marks--->450.0
--------------------------------------------------
--------------------------------------------------
Employee Object Information
--------------------------------------------------
        eno--->23
        name--->raza
        sal--->40000.0
--------------------------------------------------
--------------------------------------------------
Teacher Object Information
--------------------------------------------------
        tno--->45
        name--->kvr
        marks--->python
        exp--->30
--------------------------------------------------
```

In [20]:
```python
#Program for Demonstrating Static Method
#StaticMethodEx4
class Student:
    def getstuddata(self):
        self.sno=int(input("Enter Student  Number:"))
        self.name =input("Enter Student  Name:")
        self.marks=float(input("Enter Student  Marks:"))
class Employee:
    def getempdata(self):
        self.eno=int(input("Enter Employee  Number:"))
        self.name =input("Enter Employee  Name:")
        self.sal=float(input("Enter Employee Salary:"))
class Teacher:
    def getteacherdata(self):
        self.tno=int(input("Enter Teacher  Number:"))
        self.name =input("Enter Teacher Name:")
        self.marks=input("Enter Teacher  Subject:")
        self.exp=int(input("Enter Teacher  Exp:"))
class Hyd:
    def getobjectdata(self,objdata,objinfo): # Instnace Method
        self.dispobjectdata(objdata,objinfo)# Calling static Method w.r.t self
    @staticmethod
    def dispobjectdata(objdata,objinfo):
        print("-"*50)
        print("{} Object Information".format(objinfo))
        print("-" * 50)
        for k,v in objdata.__dict__.items():
            print("\t{}--->{}".format(k,v))
        print("-"*50)

#Main Program
s=Student()
e=Employee()
t=Teacher()
s.getstuddata()
print("--------------------------------")
e.getempdata()
```

```
print("---------------------------------")
t.getteacherdata()
#Calling Static Method of Class
h=Hyd()
h.getobjectdata(s,"Student") # Calling Instance Method w.r.t Object Name
h.getobjectdata(e,"Employee") # Calling Instance Method w.r.t Object Name
h.getobjectdata(t,"Teacher") # Calling Instancer Method w.r.t object Name
```

```
---------------------------------
---------------------------------
--------------------------------------------------
Student Object Information
--------------------------------------------------
        sno--->34
        name--->arif
        marks--->450.0
--------------------------------------------------
--------------------------------------------------
Employee Object Information
--------------------------------------------------
        eno--->23
        name--->raza
        sal--->30000.0
--------------------------------------------------
--------------------------------------------------
Teacher Object Information
--------------------------------------------------
        tno--->24
        name--->omkar
        marks--->data scince
        exp--->14
--------------------------------------------------
```

In [ ]: